Feedback Matrix Models

Equations and diagrams for `run_cleanrl_vs_mixed_dqn.py`

March 23, 2026

Abstract

This note collects the mathematical objects used in `code/run_cleanrl_vs_mixed_dqn.py`: standard CleanRL DQN, mixed backpropagation with a fixed sparse feedback matrix B, the $3F$-channel variant, and the five-head multi-critic with a trainable softmax mixture. We use one consistent index convention throughout and give both summation and matrix forms. Block diagrams summarize each forward pass and where TD errors inject into the linear feature layer.

1 Index and symbol conventions

- **Time / transition.** At step t: observation vector $\mathbf{x}_t \in \mathbb{R}^D$ (flattened state), action $a_t \in \{0, \ldots, A-1\}$ with $A = |\mathcal{A}|$, scalar reward r_t, terminal flag $d_t \in \{0, 1\}$, next observation $\mathbf{x}_{t+1}$.
- **Architecture sizes (CartPole and script defaults).** $F = 120$ linear features, $H = 84$ trunk hidden units, A actions. The script names these `OBS_TO_120`, `HIDDEN_84`.
- **Minibatch.** Training uses a set of sample indices $\mathcal{S}$; $B_{\text{bs}} = |\mathcal{S}|$ is the batch size (`BATCH_SIZE` $=$ 128). Summations $\sum_{b \in \mathcal{S}}$ are over batch elements.
- **Feature / action indices.** $i \in \{0, \ldots, F-1\}$ indexes linear features z_i. $a, a' \in \{0, \ldots, A-1\}$ index actions. For the $3F$ variant, $j \in \{0, \ldots, K-1\}$ indexes raw output channels with $K = 3F$.
- **Head index (multihead).** $m \in \{1, \ldots, M\}$ with $M = 5$.
- **Matrices.** Vectors are column vectors unless stated. $\mathbf{x}^\top$ is a row; $\mathbf{u}^\top \mathbf{v}$ is a scalar inner product. $\odot$ is elementwise (Hadamard) product.

Discount and TD target. Let $\gamma \in [0, 1)$ be the discount (`GAMMA` $= 0.99$). The target network is denoted by superscript $^-$. The scalar Bellman target used for all variants in this script is

$$y \;=\; r + \gamma\,(1 - d) \max_{a' \in \{0, \ldots, A-1\}} Q^-(\mathbf{x}_{t+1}, a'). \tag{1}$$

(Sometimes we write $Q(\mathbf{x}, a)$ when the parameter argument is implicit.)

2 Model I: CleanRL DQN (single MLP, full backprop)

2.1 Forward pass (matrix form)

Let θ collect all weights and biases. The network is

$$\mathbf{h}^{(1)} = \mathrm{ReLU}\big(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\big) \in \mathbb{R}^F, \qquad \mathbf{h}^{(2)} = \mathrm{ReLU}\big(W^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\big) \in \mathbb{R}^H, \tag{2}$$

$$\mathbf{q} = W^{(3)}\mathbf{h}^{(2)} + \mathbf{b}^{(3)} \in \mathbb{R}^A, \qquad Q(\mathbf{x}, a) = q_a = \sum_{a'=0}^{A-1} \mathbf{1}[a' = a]\, q_{a'}. \tag{3}$$

Here q_a denotes component a of $\mathbf{q}$ (0-based indexing matches the code).

2.2 TD loss and update

For a sampled transition $(\mathbf{x}, a, r, \mathbf{x}', d)$,

$$\mathcal{L}_{\mathrm{TD}} = \big(y - Q(\mathbf{x}, a)\big)^2 = \Big(y - q_a\Big)^2. \tag{4}$$

CleanRL DQN minimizes $\mathcal{L}_{\mathrm{TD}}$ with Adam on *all* layers; gradients flow through the entire MLP. Over a minibatch,

$$\mathcal{L} = \frac{1}{B_{\mathrm{bs}}} \sum_{b \in \mathcal{S}} \big(y^{(b)} - q^{(b)}_{a^{(b)}}\big)^2. \tag{5}$$

2.3 Diagram (Figure 1)

The subgraph is one feedforward chain. Dashed violet arrows denote autograd backpropagation of $\mathcal{L}_{\mathrm{TD}}$ through *all* layers. There is no separate manual B-feedback path.

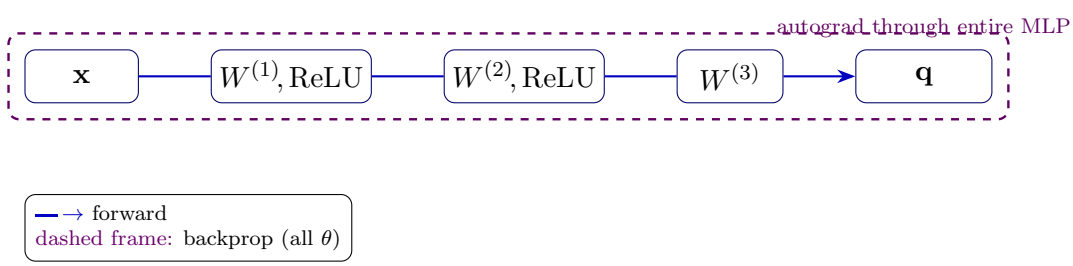

Figure 1. CleanRL DQN: one MLP; TD loss backpropagates through the full network.

3 Shared mixed-backprop setup (Models II and III)

Split the first layer from the rest:

$$\mathbf{z} = W_z\,\mathbf{x} + \mathbf{b}_z \in \mathbb{R}^F, \qquad \mathbf{h} = \mathrm{ReLU}\big(W_h\,\tilde{\mathbf{z}} + \mathbf{b}_h^{(1)}\big) \in \mathbb{R}^H, \tag{6}$$

$$\mathbf{h}' = \mathrm{ReLU}\big(W_{h2}\,\mathbf{h} + \mathbf{b}_h^{(2)}\big) \in \mathbb{R}^H, \qquad \text{readout to logits (dimension depends on variant).} \tag{7}$$

In code, the trunk sees $\tilde{\mathbf{z}} = \mathrm{stopgrad}(\mathbf{z})$ for the mixed updates: forward uses $\mathbf{z}$, but the TD loss gradient *does not* flow into $(W_z, \mathbf{b}_z)$ via the trunk. Instead, $(W_z, \mathbf{b}_z)$ are updated by a fixed linear map B applied to an error vector derived from TD error at the Q-output.

Let $\varepsilon = y - Q(\mathbf{x}, a)$ be the scalar TD error for the taken action a. The linear-layer update uses outer products with $\mathbf{x}$; for one sample,

$$W_z \leftarrow W_z + \eta_z \, \boldsymbol{\delta}_z \, \mathbf{x}^\top, \qquad \mathbf{b}_z \leftarrow \mathbf{b}_z + \eta_z \, \boldsymbol{\delta}_z, \tag{8}$$

and over a minibatch the script uses the averaged gradients

$$G_z = \frac{1}{B_{\text{bs}}} \sum_{b \in \mathcal{S}} \boldsymbol{\delta}_z^{(b)} \big(\mathbf{x}^{(b)}\big)^\top, \qquad \mathbf{g}_z = \frac{1}{B_{\text{bs}}} \sum_{b \in \mathcal{S}} \boldsymbol{\delta}_z^{(b)}, \tag{9}$$

with optional Frobenius norm clipping on G_z (max norm `FEEDBACK_GRAD_CLIP` $= 1$), then $W_z \leftarrow W_z + \eta_z G_z$, $\mathbf{b}_z \leftarrow \mathbf{b}_z + \eta_z \mathbf{g}_z$. Learning rate $\eta_z =$ `LR_LINEAR_MIXED` $= 10^{-4}$.

The trunk is trained only through $\mathcal{L}_{\text{TD}} = \varepsilon^2$ with Adam at rate `LEARNING_RATE` $= 2.5 \times 10^{-4}$.

4 Model II: Mixed backprop ($B \in \mathbb{R}^{F \times A}$)

4.1 Forward

Same as Section 3; the readout is $\mathbf{q} \in \mathbb{R}^A$ and $Q(\mathbf{x}, a) = q_a$.

4.2 Error vector and feedback

Define $\mathbf{e} \in \mathbb{R}^A$ (one-hot scaled TD error):

$$e_{a'} = \begin{cases} \varepsilon = y - Q(\mathbf{x}, a), & a' = a, \\ 0, & a' \neq a. \end{cases} \qquad \text{i.e.} \quad e_{a'} = \varepsilon \, \mathbf{1}[a' = a]. \tag{10}$$

Summation check: $\sum_{a'=0}^{A-1} e_{a'} = \varepsilon$.

The fixed matrix $B \in \mathbb{R}^{F \times A}$ has a single 1 per row:

$$B_{i,\,a'} = \mathbf{1}\big[a' \equiv i \pmod{A}\big], \qquad i = 0, \ldots, F-1. \tag{11}$$

So column a' has ones in rows $i \in \{a', a' + A, a' + 2A, \ldots\}$ (within range). The learning signal for features is

$$\boldsymbol{\delta}_z = B \, \mathbf{e} \in \mathbb{R}^F, \qquad \delta_{z,i} = \sum_{a'=0}^{A-1} B_{i,a'} \, e_{a'}. \tag{12}$$

Because $\mathbf{e}$ has only one nonzero entry at a, $\delta_{z,i} = B_{i,a} \, \varepsilon = \mathbf{1}[i \bmod A = a] \, \varepsilon$.

Implementation note. In PyTorch the code computes $\boldsymbol{\delta}_z^\top = \mathbf{e}^\top B^\top$ for row batches; equivalently $\boldsymbol{\delta}_z = B\mathbf{e}$ for column $\mathbf{e}$.

4.3 Diagram (Figure 2)

Dashed violet: autograd from $\mathcal{L}_{\text{TD}}$ through the trunk only (gradient blocked at `detach`). Dotted orange: manual update $\boldsymbol{\delta}_z = B\mathbf{e}$ to $(W_z, \mathbf{b}_z)$.

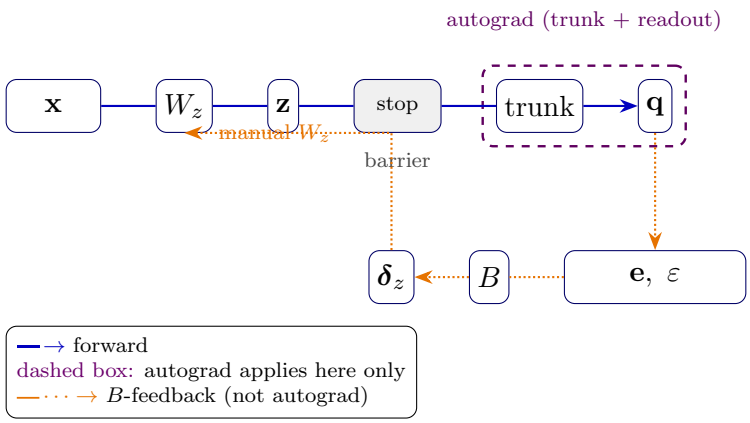

Figure 2. Mixed backprop ($B \in \mathbb{R}^{F\times A}$): trunk learns by autograd; $(W_z, \mathbf{b}_z)$ by $B\mathbf{e}$.

5 Model III: Mixed $3F$ ($B \in \mathbb{R}^{F\times K}$, $K = 3F$)

5.1 Forward: raw channels and group mean

Let $K = 3F$. The trunk outputs $\mathbf{q}_{\text{raw}} \in \mathbb{R}^K$. Assume K divisible by A and let $G = K/A$ (group size). Partition indices into action groups

$$\mathcal{G}_a = \{\, aG,\, aG+1,\, \ldots,\, (a+1)G-1 \,\}. \tag{13}$$

The Q-value for action a is the mean over its group:

$$Q(\mathbf{x}, a) = \frac{1}{G}\sum_{j\in\mathcal{G}_a} \left(\mathbf{q}_{\text{raw}}\right)_j = \frac{1}{G}\sum_{j=0}^{K-1} \mathbf{1}[j \in \mathcal{G}_a]\,\left(\mathbf{q}_{\text{raw}}\right)_j. \tag{14}$$

5.2 Error vector $\mathbf{e} \in \mathbb{R}^K$

With $\varepsilon = y - Q(\mathbf{x}, a)$ for the taken action a,

$$e_j = \begin{cases} \varepsilon/G, & j \in \mathcal{G}_a, \\ 0, & j \notin \mathcal{G}_a. \end{cases} \tag{15}$$

Then $\sum_{j\in\mathcal{G}_a} e_j = \varepsilon$ (sum of G terms each ε/G).

5.3 Feedback matrix $B \in \mathbb{R}^{F\times K}$

For each feature i, average the three channels $3i, 3i+1, 3i+2$:

$$B_{i,j} = \begin{cases} \frac{1}{3}, & j \in \{3i,\, 3i+1,\, 3i+2\}, \\ 0, & \text{otherwise}. \end{cases} \tag{16}$$

So

$$\delta_{z,i} = \sum_{j=0}^{K-1} B_{i,j}\, e_j = \frac{1}{3}\big(e_{3i} + e_{3i+1} + e_{3i+2}\big). \tag{17}$$

Matrix-vector: $\boldsymbol{\delta}_z = B\,\mathbf{e}$.

5.4 Diagram (Figure 3)

Same legend as Figure 2: dashed violet box = autograd through trunk and readout to $\mathbf{q}_{\text{raw}}$ and pooling; dotted orange = B-feedback to W_z.

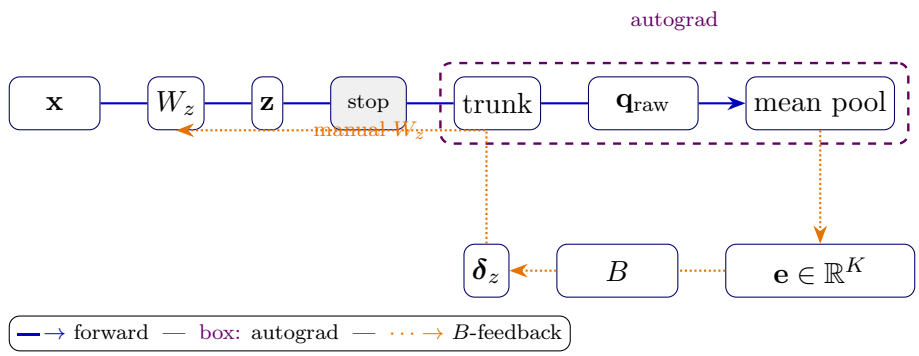

Figure 3. Mixed $3F$: readout $\mathbb{R}^K$, group mean to $Q(a)$; $\boldsymbol{\delta}_z = B\mathbf{e}$ with $B \in \mathbb{R}^{F\times K}$.

6 Model IV: Multi-head mixed network ($M = 5$ critics)

6.1 Architecture

Shared trunk on hidden $\mathbf{h} \in \mathbb{R}^H$ with five readout heads $W^{(m)} \in \mathbb{R}^{A\times H}$:

$$\mathbf{Q}^{(m)} = W^{(m)}\mathbf{h} + \mathbf{c}^{(m)} \in \mathbb{R}^A, \qquad m = 1, \ldots, 5. \tag{18}$$

Heads 1–3 use the full observation path $\mathbf{z} = W_z\mathbf{x} + \mathbf{b}_z$. Head 4 uses $\mathbf{x}$ with pole coordinates zeroed ("cart" mask); head 5 uses $\mathbf{x}$ with cart coordinates zeroed ("pole" mask). Each mask recomputes $\mathbf{z}$ through the *same* W_z.

Trainable mixture weights $\mathbf{w} = \text{softmax}(\boldsymbol{\ell}) \in \mathbb{R}^5$ with logits $\boldsymbol{\ell}$ (initialized biased toward head 1 in code). Combined Q-values:

$$Q_{\text{comb}}(\mathbf{x}, a) = \sum_{m=1}^{5} w_m\, Q^{(m)}(\mathbf{x}, a), \qquad Q^{(m)}(\mathbf{x}, a) = Q_a^{(m)}. \tag{19}$$

The policy uses $\arg\max_a Q_{\text{comb}}(\mathbf{x}, a)$.

6.2 TD targets (as in `MULTIHEAD_UNIFIED_TARGETS=True`)

Let $y^{(m)}$ be the Bellman target for head m. With unified targets, heads 1–3 share the reward return target

$$y^{(1)} = y^{(2)} = y^{(3)} = r + \gamma(1-d)\max_{a'} Q^{(1),-}(\mathbf{x}', a'), \tag{20}$$

while heads 4 and 5 still use masked forward paths but the same reward-based backup on their max:

$$y^{(4)} = r + \gamma(1-d)\max_{a'} Q^{(4),-}(\mathbf{x}', a'), \qquad y^{(5)} = r + \gamma(1-d)\max_{a'} Q^{(5),-}(\mathbf{x}', a'). \tag{21}$$

The combined head target uses the maximum over the combined Q:

$$y^{(\text{comb})} = r + \gamma(1-d)\max_{a'} Q^{-}_{\text{comb}}(\mathbf{x}', a'). \tag{22}$$

(If MULTIHEAD_UNIFIED_TARGETS is false, the code instead sets different $y^{(2)}, y^{(3)}$ using $\cos\theta'$ and γ_{short} on head 3; see script.)

Figure 4 summarizes the forward and feedback paths: three forwards share the same (W_z, trunk)—full $\mathbf{x}$ (heads 1–3), cart-masked $\mathbf{x}$ ($Q^{(4)}$), pole-masked $\mathbf{x}$ ($Q^{(5)}$). Dashed violet: autograd on $\mathcal{L}$ through trunk, heads, Q_{comb}, and $\boldsymbol{\ell}$. Dotted orange: $\mathbf{e}_{\text{heads}} \to B_{\text{mh}} \to \boldsymbol{\delta}_z$ updates W_z (with $\mathbf{z}$ stopped, autograd does not reach W_z).

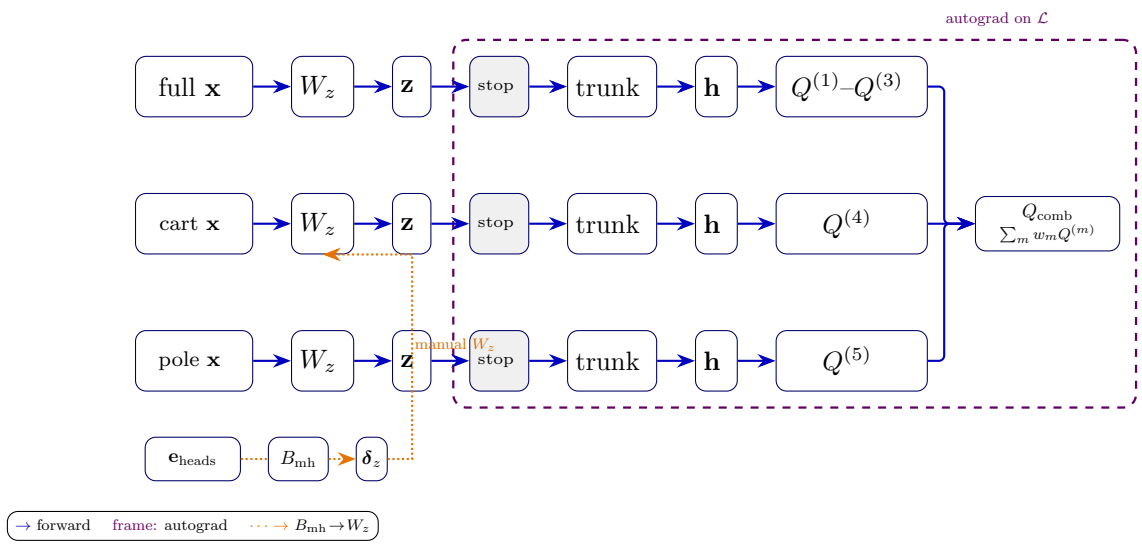

Figure 4. Multi-head (M=5): three masked pipelines; softmax-mixed Q_{comb}; manual B_{mh} feedback to shared W_z.

6.3 Per-head TD errors and linear-layer feedback

For each head, scalar error $\varepsilon^{(m)} = y^{(m)} - Q^{(m)}(\mathbf{x}, a_t)$ with the *same* sampled action a_t from the replay batch. Stack $\mathbf{e}_{\text{heads}} = (\varepsilon^{(1)}, \ldots, \varepsilon^{(5)})^\top \in \mathbb{R}^5$.

The fixed matrix $B_{\text{mh}} \in \mathbb{R}^{F \times M}$ uses the same "one hot per row modulo M" rule as make_fixed_sparse_B_5heads with *column* index $m \in \{0, \ldots, M-1\}$ matching the code:

$$(B_{\text{mh}})_{i,m} = \mathbf{1}\big[m \equiv i \pmod{M}\big], \qquad i = 0, \ldots, F-1,\ m = 0, \ldots, M-1. \tag{23}$$

Stack head errors as $\mathbf{e}_{\text{heads}} = (\varepsilon^{(1)}, \ldots, \varepsilon^{(5)})^\top$ (heads numbered as in the code). Then

$$\boldsymbol{\delta}_z = B_{\text{mh}}\, \mathbf{e}_{\text{heads}}, \qquad \delta_{z,i} = \sum_{m=0}^{M-1} (B_{\text{mh}})_{i,m}\, \varepsilon^{(m+1)}. \tag{24}$$

(Equivalently: map head index $h \in \{0, \ldots, 4\}$ to column $m = h$ so $\delta_{z,i} = \varepsilon^{(i \bmod M+1)}$ when each row has a single nonzero.)

6.4 Loss

$$\mathcal{L}_{\text{heads}} = \sum_{m=1}^{5} \big(y^{(m)} - Q^{(m)}(\mathbf{x}, a_t)\big)^2, \qquad \mathcal{L}_{\text{comb}} = \big(y^{(\text{comb})} - Q_{\text{comb}}(\mathbf{x}, a_t)\big)^2, \tag{25}$$

$$\mathcal{L} = \mathcal{L}_{\text{heads}} + \lambda_{\text{mh}}\, \mathcal{L}_{\text{comb}}, \qquad \lambda_{\text{mh}} = \texttt{MULTIHEAD_COMBINED_LOSS_WEIGHT} = 2. \tag{26}$$

Adam updates *all* parameters (including $\boldsymbol{\ell}$) on $\mathcal{L}$; the linear layer *also* receives the manual B_{mh} update as in (8), with trunk forward using detach($\mathbf{z}$) for the TD part that drives $\mathcal{L}$ (same pattern as other mixed variants).

7 Equivalence checklist (diagrams ↔ code)

- **DQN:** Equation (1) with single $\mathbf{q}$ match `CleanRLQNetwork` and MSE on gathered q_a.
- **Mixed:** $\boldsymbol{\delta}_z = B\mathbf{e}$ with one-hot $\mathbf{e}$ matches `delta_z = e @ B_fixed.t()`; manual W_z update matches `linear_feature.weight.data.add_`.
- **$3F$:** Group mean (14) matches `q_from_raw`; spread of ε/G on $\mathcal{G}_a$ matches `spread[action]`; $B_{i,3i+k} = 1/3$ matches `make_fixed_sparse_B_3f`.
- **Multihead:** Softmax mixture (19) matches `F.softmax(self.weight_logits)`; B_{mh} matches `make_fixed_sparse_B_5heads`; stacked errors match `e = torch.cat([e1,...,e5], dim=1)`.

8 Comparison

Table 1. How each architecture trains the linear feature layer W_z vs. the trunk (readout).

Model	**Autograd to W_z?**	**Manual B to W_z?**	**Trunk**	**Feedback shape**
CleanRL DQN	yes	no	full MLP	—
Mixed (A)	no	yes ($B \in \mathbb{R}^{F\times A}$)	trunk + readout	one-hot TD on a
Mixed $3F$	no	yes ($B \in \mathbb{R}^{F\times K}$)	trunk + K-dim readout	spread TD on group $\mathcal{G}_a$
Multihead	no	yes ($B_{\text{mh}} \in \mathbb{R}^{F\times 5}$)	shared trunk + 5 heads + softmax	stacked head errors

The table contrasts how W_z is trained: only mixed variants (including multi-head, Figure 4) use manual B-feedback; CleanRL DQN relies on autograd into W_z through the full MLP.